

ADMISSIONUM

Guía didáctica completa

De un proyecto Spring Boot para principiantes a un despliegue completo en Azure

Estado documentado

Rama main del repositorio Franer-Narf/Admissionum fijada en el commit 5233f9d5b6c73c362af4cb2239852171b53fc796. La trazabilidad v20 se interpreta como un documento acumulativo que incorpora las fases anteriores y añade las sucesivas versiones.

Edición didáctica - septiembre de 2026

Sin secretos, credenciales reales ni datos personales de invitados

Nota de fuentes y criterio de prioridad

Este documento se ha redactado con una regla de prioridad explícita. Primero se explica el código actual de `main`; después se utilizan los documentos actuales del repositorio; a continuación se consulta `traceability v20.txt` para reconstruir la evolución histórica; por último se emplea conocimiento técnico general únicamente para explicar conceptos como HTTP, JPA, Docker, CI/CD u OIDC. Si una descripción histórica contradice el código actual, prevalece el código actual.

La trazabilidad v20 no se considera una instantánea aislada. Es un documento acumulativo: contiene el Documento Maestro, decisiones, incidencias, correcciones y cierres de las fases anteriores, y después añade las fases siguientes. Por tanto, sirve como registro histórico completo para comprender por qué el proyecto llegó a su forma actual.

El documento diferencia cuatro tipos de información: estado actual, evolución histórica, ejemplos didácticos y pseudocódigo. Los ejemplos con nombres, correos, contraseñas, identificadores, servidores o direcciones son ficticios. Nunca deben copiarse como secretos de producción.

Regla práctica

Cuando veas una ruta como `src/main/...`, se refiere a un archivo real del proyecto. Cuando el texto diga “ejemplo educativo” o “pseudocódigo”, no se está afirmando que ese fragmento exista literalmente en el repositorio.

Índice

1. 1. Portada
2. 2. Cómo utilizar esta guía
3. 3. El problema que resuelve Admitionum
4. 4. Visión general de la arquitectura
5. 5. Preparación del entorno
6. 6. Estructura del proyecto
7. 7. Modelo de datos
8. 8. Spring Data JPA
9. 9. DTO y validación
10. 10. API REST pública
11. 11. InvitationService y lógica de negocio
12. 12. Transacciones y rollback
13. 13. Controladores Spring MVC
14. 14. Frontend público
15. 15. Panel administrativo
16. 16. Spring Security
17. 17. Gestión de errores
18. 18. Pruebas automatizadas
19. 19. H2 y ejecución local
20. 20. Azure SQL Database
21. 21. Docker
22. 22. Azure Container Registry
23. 23. Azure Container Apps
24. 24. GitHub Actions y CI/CD

- 25. 25. Azure OIDC
- 26. 26. Git y flujo de trabajo
- 27. 27. Seguridad y secretos
- 28. 28. Evolución histórica del proyecto
- 29. 29. Fase 20 como caso práctico de evolución de requisitos
- 30. 30. Cómo reproducir un proyecto semejante
- 31. 31. Errores frecuentes y resolución de problemas
- 32. 32. Glosario
- 33. 33. Conclusiones
- 34. 34. Apéndices

1. Portada

La portada identifica el propósito del material: convertir un proyecto real y su evolución en un recorrido formativo. Admitionum no se presenta como una colección de tecnologías desconectadas, sino como un sistema completo cuyo valor pedagógico está en observar cómo una petición del navegador atraviesa controlador, servicio, repositorio y base de datos, y cómo ese mismo sistema se empaqueta y despliega en Azure.

Lectura recomendada

Si partes de cero, avanza en orden. Si ya conoces Spring Boot, puedes saltar a los capítulos 9-18 para estudiar API, seguridad y pruebas, y después a los capítulos 20-25 para infraestructura y CI/CD.

2. Cómo utilizar esta guía

La guía está pensada para una persona con conocimientos iniciales. Cada capítulo combina definición, ubicación en el proyecto, uso real, interacción con otros componentes, consecuencias de eliminarlo, errores típicos y una forma de comprobar el comportamiento.

Convenciones visuales

Etiqueta	Significado
Estado actual	Describe lo que existe en main.
Evolución histórica	Explica cómo era una fase anterior.
Concepto clave	Idea técnica que conviene retener.
Error común	Fallo frecuente y su causa.
Cómo verificar	Prueba manual o automatizada para confirmar el comportamiento.

Los comandos se presentan con shell, directorio, finalidad, resultado esperado y comprobación. Los comandos destructivos deben leerse dos veces antes de ejecutarlos. En esta guía no se incluyen secretos ni órdenes que modifiquen recursos reales de Azure de forma irreversible.

3. El problema que resuelve Admitionum

Admitionum gestiona confirmaciones de asistencia a una boda. El invitado abre un formulario público, indica si asistirá, cuántas personas irán y datos adicionales; el administrador consulta las respuestas en un panel protegido y puede exportarlas a CSV. El proyecto también sirve como ejemplo de una aplicación web completa que integra backend, frontend, persistencia, seguridad, contenedores, nube y automatización.

Problema funcional

Sin una aplicación, las confirmaciones pueden llegar por canales distintos y ser difíciles de consolidar. Admitionum convierte cada respuesta en datos estructurados y aplica reglas en el backend para que el número de asistentes sea coherente con la decisión de asistencia.

Cambio de Fase 20

El requisito más importante para interpretar el sistema actual es que el invitado ya no necesita un código. El acceso principal es un QR o enlace común. Cada envío correcto crea una Invitation interna y su RsvpResponse. Los endpoints basados en AccessCode siguen presentes para compatibilidad con el diseño anterior, pero no son el viaje principal de usuario.

Evolución del flujo de invitaciones

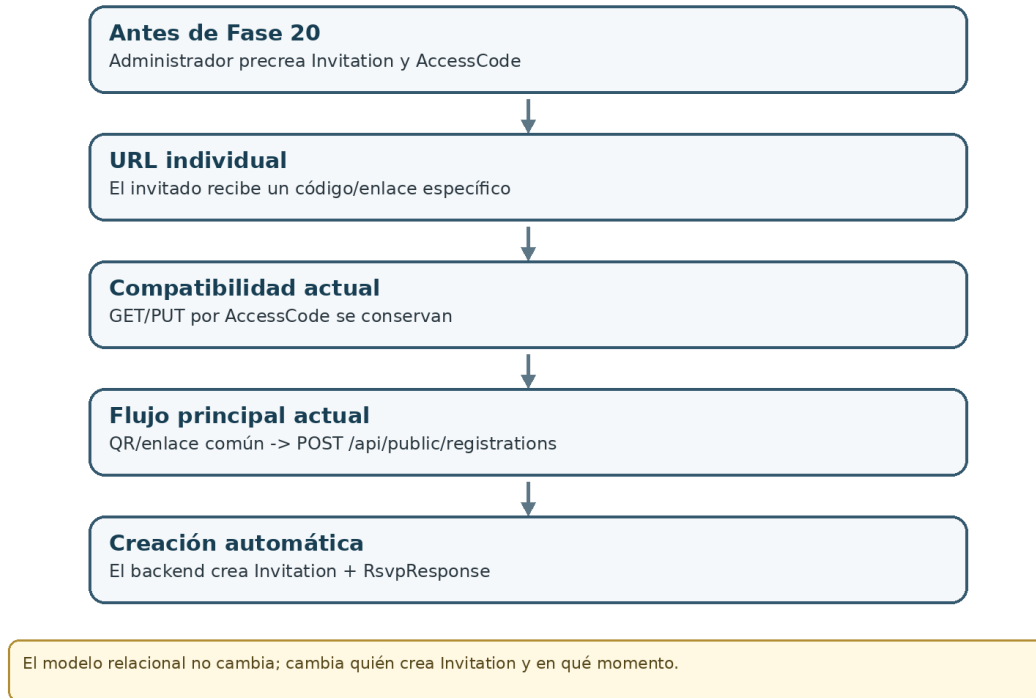


Figura 1. Del modelo de invitaciones individuales al registro público general.

4. Visión general de la arquitectura

Admissionum es una aplicación monolítica Spring Boot. “Monolítica” no significa “mal diseñada”: aquí significa que el frontend estático, la API, la seguridad y la lógica de negocio se empaquetan y despliegan juntos. Para el tamaño y objetivo del proyecto, esta decisión reduce piezas operativas sin impedir una separación interna por capas.

Arquitectura general de Admitionum

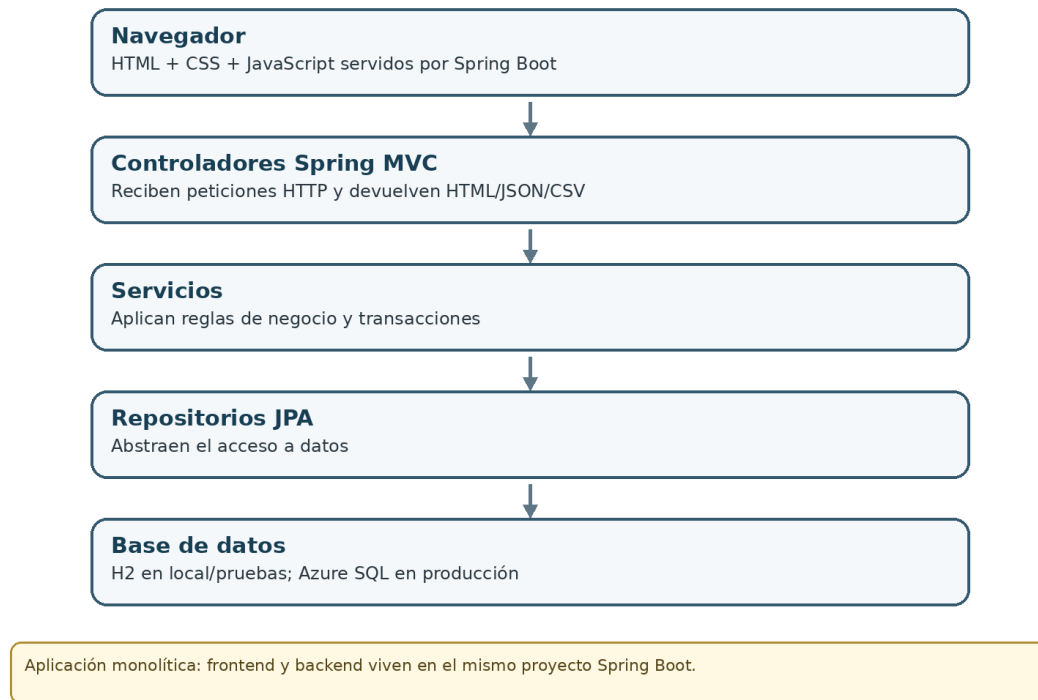


Figura 2. Arquitectura general de la aplicación.

Qué es frontend y qué es backend

El frontend es lo que se ejecuta en el navegador: HTML define la estructura, CSS la presentación y JavaScript el comportamiento. El backend es el proceso Java/Spring Boot: recibe HTTP, valida, aplica reglas, lee o escribe datos y devuelve respuestas. El frontend nunca se conecta directamente a la base de datos.

HTTP y API

HTTP es el protocolo usado por navegador y servidor. Una API es un contrato de comunicación: combina rutas, métodos, cuerpos y respuestas. En Admitionum, `POST /api/public/registrations` es el contrato principal para registrar una respuesta general.

Capas internas de Spring Boot



Figura 3. Capas de una petición dentro de Spring Boot.

5. Preparación del entorno

El proyecto exige Java 25 y utiliza Maven Wrapper, por lo que no es necesario instalar Maven globalmente. Git permite clonar y trabajar con ramas. Docker es necesario para construir o ejecutar la imagen de contenedor; Azure CLI solo es necesario cuando se trabaja manualmente con Azure.

Requisitos mínimos

Herramienta	Uso en el proyecto	Comprobación
Java 25	Compilar y ejecutar Spring Boot	<code>java -version</code>
Git	Clonar, ramas, commits y PR	<code>git --version</code>
Maven Wrapper	Compilar/pruebas sin Maven global	<code>./mvnw -version</code> o <code>.\mvnw.cmd -version</code>
Docker	Imagen y contenedor	<code>docker version</code>
Azure CLI	Operaciones manuales en Azure	<code>az version</code>

Elemento	Detalle
Shell / sistema	PowerShell (Windows)
Directorio de trabajo	raíz de Admitionum

Elemento	Detalle
Comando	.\mvnw.cmd clean verify
Propósito	Ejecutar toda la batería de pruebas y empaquetar.
Resultado esperado	BUILD SUCCESS.
Errores frecuentes	JAVA_HOME incorrecto, descarga de dependencias bloqueada, puerto irrelevante porque verify no inicia el servidor.
Cómo verificar	El proceso termina con código 0 y se crea el JAR en target/.

Elemento	Detalle
Shell / sistema	Git Bash / Linux / macOS
Directorio de trabajo	raíz de Admissionum
Comando	./mvnw clean verify
Propósito	Misma verificación con shell tipo Unix.
Resultado esperado	BUILD SUCCESS.
Errores frecuentes	Permiso de ejecución de mvnw o Java 25 no disponible.
Cómo verificar	`echo \$?` devuelve 0.

6. Estructura del proyecto

La estructura separa responsabilidades. Los paquetes `controller`, `service`, `repository`, `model`, `dto`, `config` y `exception` expresan una arquitectura por capas. Los recursos contienen configuración y frontend. Las pruebas reflejan las mismas responsabilidades y añaden escenarios de integración.

Ruta	Responsabilidad
pom.xml	Dependencias, Java 25, Spring Boot 4.1.0 y plugins Maven.
src/main/java/nc/admissionum/controller	Endpoints HTTP.
src/main/java/nc/admissionum/service	Reglas de negocio.
src/main/java/nc/admissionum/repository	Acceso a datos con Spring Data JPA.
src/main/java/nc/admissionum/model	Entidades JPA.
src/main/java/nc/admissionum/dto	Contratos de entrada/salida.
src/main/java/nc/admissionum/config	Spring Security y configuración local específica.
src/main/resources/static	HTML/CSS/JS público y administrativo.
database/schema.sql	Esquema SQL Server/Azure SQL.
src/test/java	Pruebas unitarias, MVC, JPA, seguridad e integración.
.github/workflows/deploy.yml	CI/CD.
Dockerfile	Imagen de ejecución.

Error común

Confundir la estructura de paquetes con microservicios. Son paquetes dentro del mismo artefacto y del mismo proceso Java; el sistema sigue siendo monolítico.

7. Modelo de datos

El modelo tiene dos entidades principales: Invitation y RsvpResponse. La relación es uno a cero-o-uno: puede existir una invitación sin respuesta, pero no dos respuestas para la misma invitación. En el flujo general de Fase 20 ambas se crean casi simultáneamente, mientras que el flujo de compatibilidad todavía puede tener invitaciones pendientes.

Relación de datos

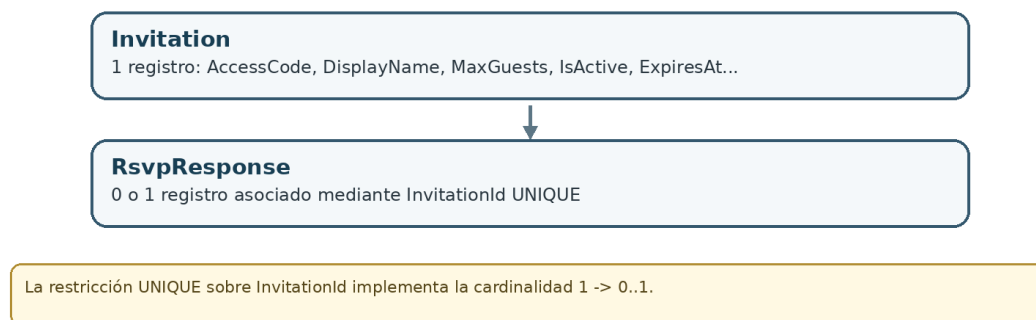


Figura 4. Cardinalidad entre Invitation y RsvpResponse.

Entidad Java	Tabla SQL	Campos destacados
Invitation	Invitations	Id, AccessCode, DisplayName, MaxGuests, IsActive, ExpiresAt, CreatedAt, UpdatedAt
RsvpResponse	RsvpResponses	Id, InvitationId, GuestName, Contact, AttendanceConfirmed, AttendeeCount, Intolerances, AdditionalComment, SubmittedAt, UpdatedAt

Por qué Fase 20 no exigió rediseñar la base

La tabla Invitations ya admitía crear una invitación en cualquier momento y la tabla RsvpResponses ya enlazaba una respuesta mediante InvitationId. La restricción UNIQUE sobre InvitationId seguía siendo válida. El código general genera un AccessCode interno 'REG-' + UUID, que cabe dentro de VARCHAR(50), establece MaxGuests=20 y crea la respuesta asociada. Cambió el momento de creación, no la estructura relacional.

Fragmento real: database/schema.sql

```

CONSTRAINT UQ_RsvpResponses_InvitationId UNIQUE (InvitationId),
CONSTRAINT FK_RsvpResponses_Invitations FOREIGN KEY (InvitationId) REFERENCES dbo.Invitations(Id)
  
```

8. Spring Data JPA

JPA permite trabajar con objetos Java y dejar que el framework traduzca operaciones a SQL. Una entidad es un objeto persistente; un repositorio es una interfaz de acceso; Hibernate es la implementación ORM utilizada por Spring Data JPA.

Entidad frente a tabla

`Invitation` no es “la tabla”: es una representación Java mapeada a `Invitations`. Anotaciones como `@Entity`, `@Table`, `@Id`, `@Column` y `@OneToOne` describen el mapeo. El esquema SQL añade restricciones que siguen protegiendo los datos incluso si una futura ruta de aplicación omitiera una validación.

Fragmento real: RsvpResponse.java

```
@OneToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "InvitationId", nullable = false, unique = true)
private Invitation invitation;
```

Repositorios

Los repositorios evitan escribir JDBC manual para operaciones comunes. Métodos como `findById(...)`, `findByAccessCode(...)`, `saveAndFlush(...)`, `findAll()` o `count()` permiten expresar la intención. La lógica de negocio, sin embargo, no debe mudarse al repositorio: corresponde al servicio.

Concepto clave

JPA simplifica la persistencia, pero no sustituye el diseño de base de datos. Las restricciones UNIQUE, FK y CHECK siguen siendo parte esencial de la integridad.

9. DTO y validación

Un DTO es un objeto creado para transportar datos entre capas o a través de una API. Admissionum evita devolver directamente entidades JPA; de esta forma puede decidir qué datos son públicos y qué campos internos deben permanecer ocultos.

SaveRsvpRequest

Campo	Regla Bean Validation	Regla de negocio adicional
guestName	@NotBlank, @Size(2..200)	Se recorta con trim.
contact	@NotBlank, @Size(3..200)	Se recorta con trim.
attendanceConfirmed	@NotNull	Define la regla condicional de attendeeCount.
attendeeCount	@NotNull, @Min(0), @Max(20)	Si asiste: >=1; si no: exactamente 0.
intolerances	@Size(max=500)	Cadena vacía se normaliza a null.
additionalComment	@Size(max=1000)	Cadena vacía se normaliza a null.

Fragmento real: SaveRsvpRequest.java

```
@NotNull
@Min(0)
@Max(20)
private Integer attendeeCount;
```

Bean Validation resuelve restricciones locales de cada campo. La relación entre dos campos -por ejemplo, “si attendanceConfirmed es false, attendeeCount debe ser 0”- se aplica en el servicio. Esta separación evita forzar reglas condicionales complejas dentro del DTO.

Error común

Confiar únicamente en validaciones HTML o JavaScript. Un cliente puede saltarse el navegador y llamar a la API directamente; por eso el backend es la autoridad final.

10. API REST pública

La API pública es el punto de entrada para invitados. REST aquí se usa de manera práctica: rutas basadas en recursos, métodos HTTP y JSON. El endpoint principal de Fase 20 es `POST /api/public/registrations`.

Catálogo

Método	Endpoint	Uso
GET	/api/public/health	Comprobar que la aplicación responde.
POST	/api/public/registrations	Flujo general actual: crea Invitation + RsvpResponse.
GET	/api/public/invitations/{code}	Compatibilidad: consultar una invitación existente.
PUT	/api/public/invitations/{code}/response	Compatibilidad: crear o actualizar la respuesta de una invitación.

Ejemplo educativo de petición

```
POST /api/public/registrations
Content-Type: application/json
```

```
{
  "guestName": "Ana Ejemplo",
  "contact": "ana@example.test",
  "attendanceConfirmed": true,
  "attendeeCount": 3,
  "intolerances": "Sin gluten",
  "additionalComment": "Llegaremos el viernes"
}
```

Ejemplo educativo de respuesta

```
HTTP/1.1 201 Created
```

```
{
  "success": true,
  "message": "Tu respuesta se ha guardado correctamente.",
  "updatedAt": "2027-04-01T17:30:00"
}
```

Estado actual

El POST general no devuelve ni ID de Invitation, ni ID de RsvpResponse ni AccessCode generado. Es una frontera de datos deliberada.

11. InvitationService y lógica de negocio

`InvitationService` concentra las reglas principales. Está anotado con `@Service` y, a nivel de clase, `@Transactional(readOnly = true)`. Los métodos que escriben datos declaran su propia transacción de escritura con `@Transactional`.

Flujo registerPublicResponse

Flujo público principal después de Fase 20

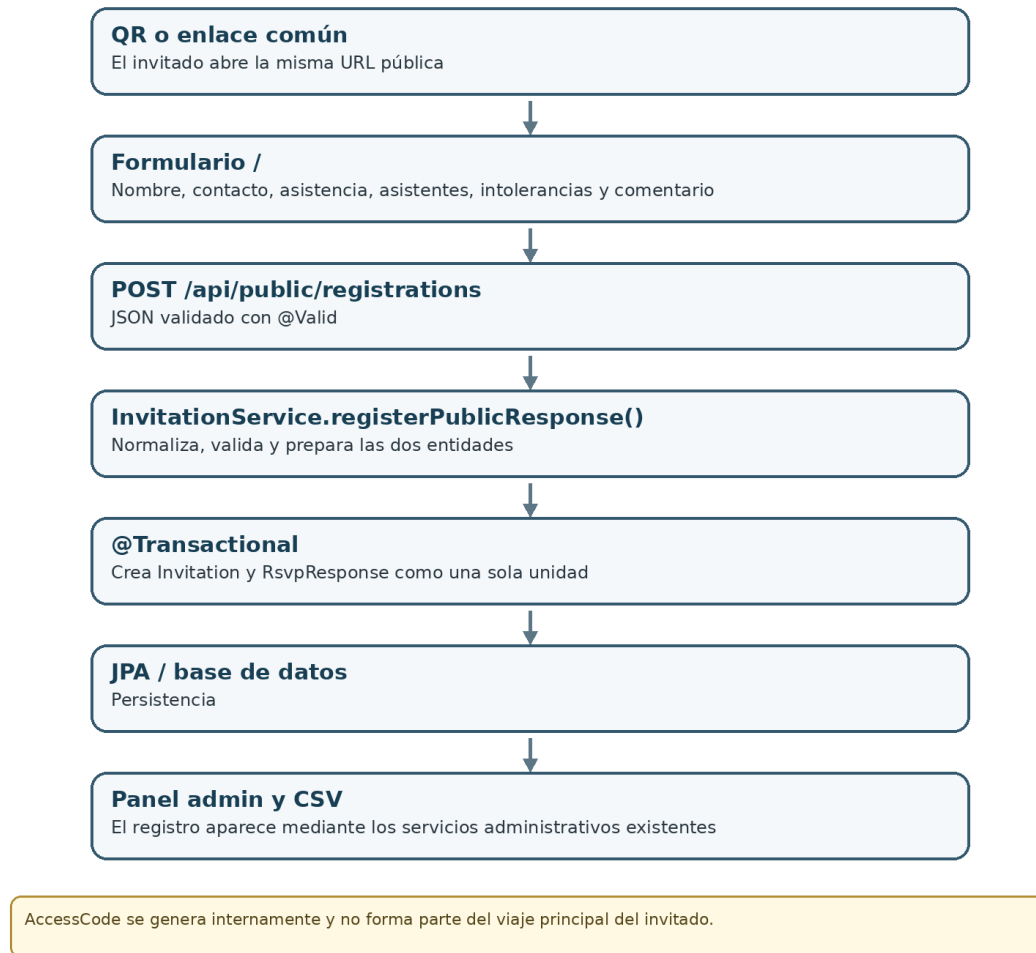


Figura 5. Recorrido completo de un registro público general.

El método limpia nombre y contacto, construye una `Invitation` con AccessCode aleatorio, DisplayName igual al nombre enviado, MaxGuests=20, IsActive=true y ExpiresAt=null; valida el número de asistentes; guarda y fuerza el flush de la invitación; crea la respuesta; guarda y fuerza el flush; finalmente devuelve un DTO de confirmación.

Fragmento real: InvitationService.registerPublicResponse()

```
Invitation invitation = new Invitation(
    generateAccessCode(),
    guestName,
    GENERAL_MAX_GUESTS,
    true,
    null
);
```

Fragmento real: generación del AccessCode interno

```
return REGISTRATION_CODE_PREFIX + UUID.randomUUID();
```

Normalización

`clean` aplica `trim()` y `cleanNullable` transforma cadenas vacías en `null`. Esto evita almacenar espacios accidentales y simplifica la interpretación de campos opcionales.

12. Transacciones y rollback

Una transacción agrupa varias operaciones como una única unidad lógica. La propiedad que interesa aquí es atomicidad: o se guardan las dos entidades o no se guarda ninguna. Sin este comportamiento, un fallo en la segunda escritura podría dejar una Invitation sin RsvpResponse creada por un formulario que el usuario cree que falló.

Atomicidad del registro general

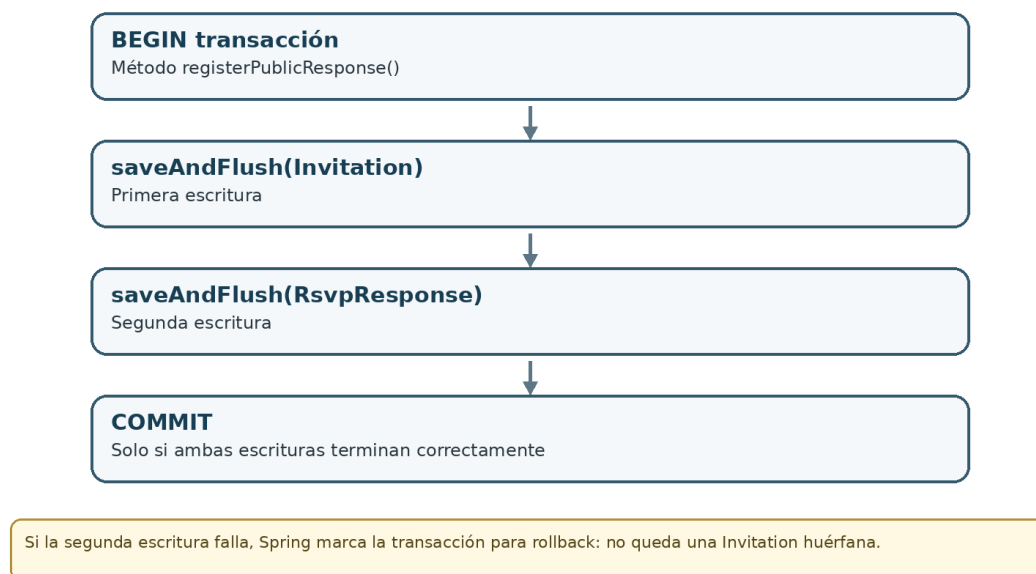


Figura 6. Commit y rollback del registro general.

Por qué se usa saveAndFlush

`saveAndFlush` solicita guardar y sincronizar con la base antes de continuar. En la prueba de rollback, el repositorio de respuestas se sustituye por un mock que lanza una excepción en esa segunda escritura. Spring revierte la transacción y el recuento de invitaciones queda igual que antes.

Idea real comprobada por PublicRegistrationTransactionTest

```
long invitationsBefore = invitationRepository.count();
// la segunda escritura falla
long invitationsAfter = invitationRepository.count();
assertThat(invitationsAfter).isEqualTo(invitationsBefore);
```

Cómo verificar

Ejecuta `PublicRegistrationTransactionTest`. Si el rollback no funcionara, la aserción del recuento detectaría una Invitation huérfana.

13. Controladores Spring MVC

Los controladores adaptan HTTP a llamadas Java. `@RestController` indica que el valor devuelto se serializa como respuesta; `@RequestMapping` define la ruta base; `@PostMapping`, `@GetMapping` y `@PutMapping` el método; `@RequestBody` lee JSON; `@Valid` activa Bean Validation; `ResponseEntity` permite fijar estado y cuerpo.

Fragmento real simplificado del controlador

```
@RestController
@RequestMapping("/api/public/registrations")
public class PublicRegistrationController {
    @PostMapping
    public ResponseEntity<SaveRsvpResponse> register(
        @Valid @RequestBody SaveRsvpRequest request) {
        SaveRsvpResponse response = invitationService.registerPublicResponse(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }
}
```

El controlador no construye entidades ni contiene SQL. Su responsabilidad termina al validar el contrato HTTP, delegar en el servicio y escoger la respuesta HTTP. Esta disciplina facilita probar la capa web de manera aislada con MockMvc.

14. Frontend público

El frontend público utiliza HTML, CSS y JavaScript sin frameworks. Este diseño mantiene el proyecto accesible para principiantes: el DOM representa la página, los eventos reaccionan a la interacción, `fetch` envía la solicitud y JSON transporta los datos.

Flujo del navegador

El usuario abre `/`, completa el formulario y el JavaScript reúne los valores. Después crea un objeto JSON y hace `POST /api/public/registrations`. Si el servidor devuelve un error de validación, la interfaz puede mostrar mensajes; si devuelve 201, presenta la confirmación. La validación visual del navegador mejora experiencia, pero no reemplaza la del backend.

Pseudocódigo educativo equivalente al patrón usado por public-app.js

```
fetch("/api/public/registrations", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload)
});
```

Fase 20

El JavaScript actual ya no lee un código desde la URL ni precarga una Invitation. La página es el formulario general que puede enlazarse desde un QR común.

15. Panel administrativo

El área administrativa presenta una visión consolidada de las invitaciones y respuestas. Está protegida por Spring Security y utiliza los servicios existentes para calcular estadísticas, listar registros y exportar CSV.

Qué muestra

Función	Descripción
Dashboard	Totales, respondidas, pendientes, confirmadas, rechazadas, asistentes confirmados e intolerancias.
Listado	Nombre de invitación, invitado, contacto, estado, asistentes y campos opcionales.
Búsqueda/filtros	Aplicados en el frontend administrativo sobre los datos recuperados.
CSV	Exporta el conjunto administrativo con neutralización de posibles fórmulas.

Una consecuencia importante de Fase 20 es que el panel no necesitó un rediseño funcional para ver los nuevos registros. `AdminService` ya opera sobre las entidades y sus relaciones; como el flujo general crea las mismas entidades, aparecen de forma natural en dashboard, listado y CSV.

16. Spring Security

Spring Security separa zona pública y zona administrativa. El proyecto utiliza autenticación de formulario y sesión HTTP. No utiliza JWT. Autenticación responde “quién eres”; autorización responde “qué puedes hacer”.

Frontera de seguridad

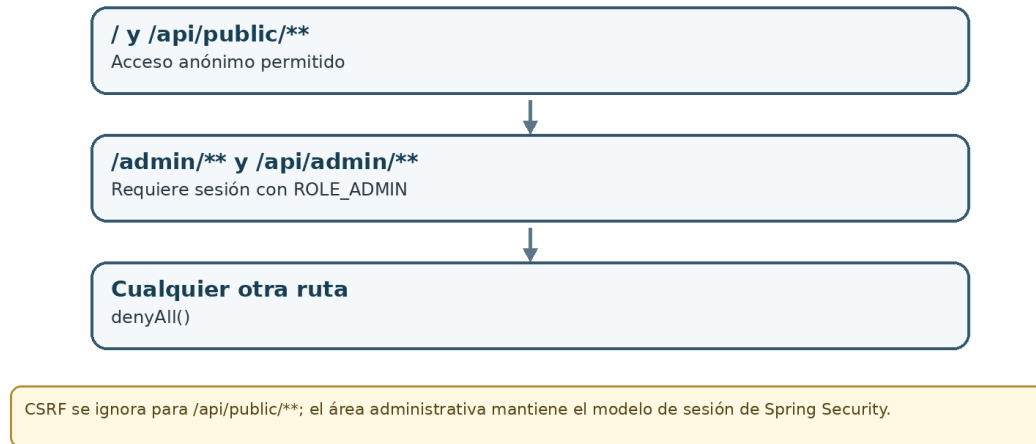


Figura 7. Reglas de acceso principales.

Ruta	Acceso
/	Público
/css/**, /js/**	Público
/api/public/**	Público
/admin/**	ROLE_ADMIN
/api/admin/**	ROLE_ADMIN
resto	denyAll

Fragmento real: SecurityConfig.java

```
.requestMatchers("/admin/**", "/api/admin/**").hasRole("ADMIN")
.anyRequest().denyAll()
```

CSRF

CSRF protege acciones autenticadas basadas en cookies frente a solicitudes forjadas desde otros sitios. La configuración ignora CSRF para `/api/public/\*\*` porque esos endpoints son anónimos y el formulario general debe poder hacer POST sin token CSRF. Esto no convierte la API pública en privada ni evita spam; es una decisión de modelo de acceso.

17. Gestión de errores

Una API útil no solo devuelve datos cuando todo va bien; también devuelve errores previsibles y coherentes. Admissionum centraliza respuestas mediante un manejador global y códigos como `VALIDATION\_ERROR`, `INVALID\_GUEST\_COUNT`, `INVITATION\_NOT\_FOUND`, `INVITATION\_DISABLED` e `INVITATION\_EXPIRED`.

Dos niveles de error

Bean Validation detecta campos ausentes, vacíos o fuera de tamaño antes de ejecutar el servicio. La lógica de negocio detecta combinaciones semánticamente inválidas, como confirmar asistencia con cero asistentes. El flujo de compatibilidad añade errores relacionados con la existencia, estado y caducidad de la invitación.

Ejemplo educativo compatible con docs/api.md

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Revisa los campos indicados.",
    "fields": { "contact": "El contacto es obligatorio." }
  }
}
```

Error común

Devolver la traza de una excepción al usuario. Las trazas son útiles en logs, pero pueden revelar detalles internos. La respuesta pública debe ser controlada.

18. Pruebas automatizadas

Las pruebas de Admitionum cubren distintos niveles. Las unitarias verifican reglas aisladas; las de MVC comprueban contratos HTTP; las JPA validan el mapeo; las de seguridad comprueban autorización; las de integración recorren varias capas con una base de datos de prueba.

Prueba	Riesgo que cubre
InvitationServiceTest	Reglas de invitaciones, creación/actualización y validación del servicio.
PublicRegistrationControllerTest	201, validación 400 y delegación del POST general.
SecurityConfigTest	Público anónimo, admin protegido, roles y CSRF público.
PublicRegistrationFlowTest	Persistencia completa, AccessCode REG-, MaxGuests=20, visibilidad admin y CSV.
PublicRegistrationTransactionTest	Rollback si falla la segunda escritura.

Qué demuestra PublicRegistrationFlowTest

La prueba integra servicio, repositorios, AdminService y CsvExportService. Verifica que la Invitation generada empiece por `REG-`, tenga 40 caracteres, use el nombre enviado como DisplayName, MaxGuests=20, esté activa y no tenga caducidad; después comprueba la RsvpResponse, el estado administrativo `CONFIRMED` y la presencia de los datos en CSV.

Concepto clave

Una prueba de integración no sustituye las unitarias y viceversa. Cada nivel localiza tipos de regresión diferentes.

19. H2 y ejecución local

H2 reduce la fricción local. El perfil por defecto es `local` y usa una base persistente en `./data/admissionum`. Las pruebas usan otro perfil y una base aislada en memoria, de modo que ejecutar tests no debe contaminar los datos locales.

Perfiles y bases de datos

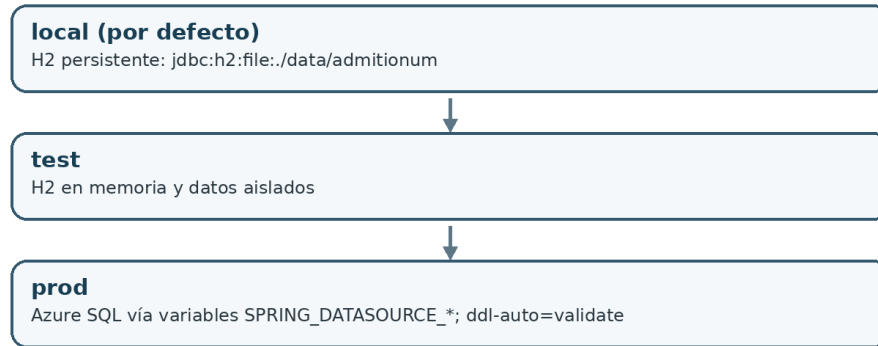


Figura 8. Diferencia entre local, test y producción.

Configuración real resumida

```

spring.profiles.default=local
spring.datasource.url=jdbc:h2:file:./data/admissionum
spring.jpa.hibernate.ddl-auto=update
  
```

Elemento	Detalle
Shell / sistema	PowerShell
Directorio de trabajo	raíz del proyecto
Comando	\$env:ADMIN_USERNAME="admin"; \$env:ADMIN_PASSWORD="local-only"; .\mvnw.cmd spring-boot:run
Propósito	Iniciar el perfil local con credenciales ficticias.
Resultado esperado	Servidor en puerto 8080 y H2 persistente.
Errores frecuentes	Variables ADMIN_* ausentes, puerto 8080 ocupado, Java incorrecto.
Cómo verificar	Abrir `http://localhost:8080/api/public/health` y comprobar estado ok.

Seguridad

La contraseña del ejemplo es deliberadamente ficticia. No la reutilices y no la escribas en archivos versionados.

20. Azure SQL Database

Producción usa Microsoft SQL Server mediante Azure SQL Database. El perfil `prod` no contiene una cadena real: obtiene URL, usuario y contraseña de variables de entorno y usa `spring.jpa.hibernate.ddl-auto=validate`, de modo que Hibernate comprueba el esquema en lugar de modificarlo automáticamente.

application-prod.properties

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
```

La conexión de producción debe usar cifrado y la red/firewall de Azure debe permitir el origen necesario. El esquema se crea o gestiona de forma explícita con `database/schema.sql`. Separar credenciales del repositorio evita que el código fuente se convierta en almacén de secretos.

Problemas frecuentes

Síntoma	Causa probable	Comprobación
Login failed	Usuario/contraseña o permisos SQL	Revisar variables y usuario de aplicación.
Connection timeout	Firewall, servidor pausado o red	Comprobar estado del servidor y reglas de conectividad.
Schema-validation error	Tabla/columna/restricción incompatible con entidades	Comparar database/schema.sql con anotaciones JPA.

21. Docker

Docker empaqueta el JAR y su runtime Java en una imagen reproducible. Un Dockerfile describe cómo construir esa imagen; una imagen es el artefacto; un contenedor es una instancia en ejecución. Son conceptos distintos.

De código a contenedor

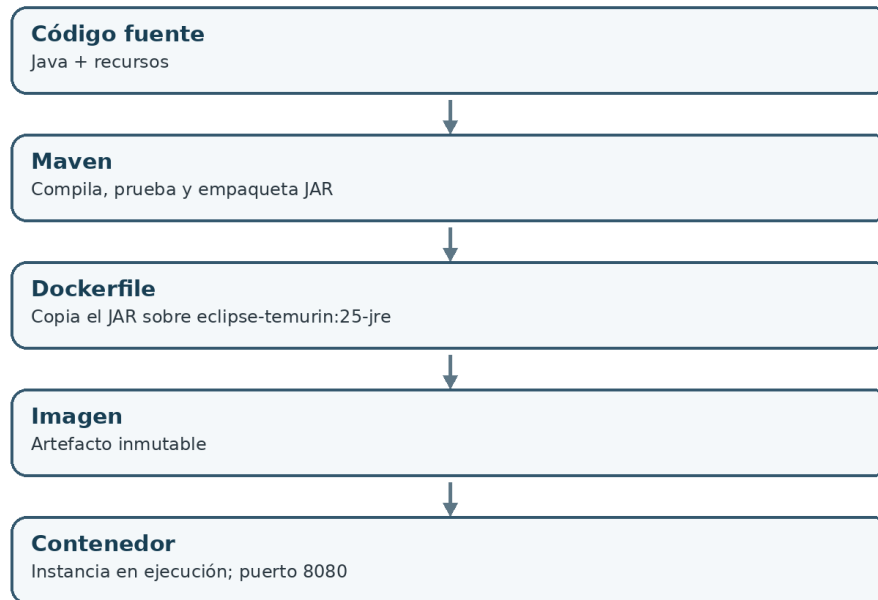


Figura 9. Ciclo código -> JAR -> imagen -> contenedor.

Dockerfile actual completo

```

FROM eclipse-temurin:25-jre
WORKDIR /app
COPY target/admissionum-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
  
```

Elemento	Detalle
Shell / sistema	Git Bash / PowerShell con Docker
Directorio de trabajo	raíz del proyecto
Comando	docker build -t admissionum .
Propósito	Crear una imagen local usando el JAR ya empaquetado.
Resultado esperado	Imagen `admissionum` construida.
Errores frecuentes	No existe el JAR en target/, Docker Desktop apagado, descarga de imagen base bloqueada.
Cómo verificar	`docker image ls admissionum` muestra la imagen.

El Dockerfile actual es deliberadamente sencillo y de una sola etapa. Una construcción multi-stage sería una posible mejora futura, pero no debe describirse como si fuera la implementación actual.

22. Azure Container Registry

Azure Container Registry (ACR) es el registro privado de imágenes Docker. Su función es almacenar versiones identificables del contenedor que después puede ejecutar Azure Container Apps. En CI/CD, la etiqueta usa `github.sha`, lo que relaciona una imagen con un commit concreto.

Por qué no basta con “tener Docker”

Docker construye y ejecuta imágenes localmente, pero producción necesita un lugar desde el que el servicio pueda obtenerlas. ACR cumple esa función. GitHub Actions inicia sesión en Azure y después en ACR, hace `docker push` y entrega la referencia completa de imagen a Container Apps.

Trazabilidad

Una etiqueta basada en el SHA evita depender exclusivamente de `latest`. Permite saber qué código produjo la imagen desplegada.

23. Azure Container Apps

Azure Container Apps ejecuta la imagen en un servicio gestionado. Conceptos esenciales son aplicación, revisión, réplica, ingress, variables de entorno y referencia de imagen. Una actualización de imagen crea o activa una nueva revisión según la configuración.

Relación GitHub - ACR - Container Apps - Azure SQL

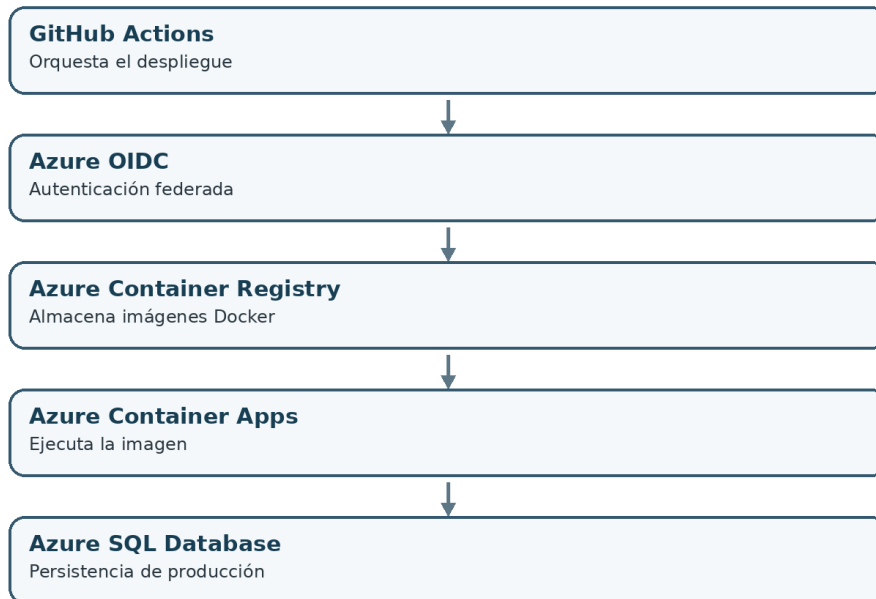


Figura 10. Relación entre los componentes de despliegue.

El workflow obtiene el FQDN público, consulta la imagen configurada y llama repetidamente a `/api/public/health`. Esa verificación no demuestra por sí sola que todas las operaciones de negocio funcionen, pero sí confirma que la revisión responde y que el endpoint básico está disponible.

Diagnóstico

Problema	Qué revisar
La revisión no arranca	Logs, variables, acceso a ACR, puerto 8080, conexión a Azure SQL.
Health no responde	Ingress/FQDN, estado de revisión, arranque de Spring Boot.
Imagen incorrecta	Resultado de `az containerapp show` frente a IMAGE_REFERENCE.

24. GitHub Actions y CI/CD

CI significa integración continua: cada cambio se prueba de forma repetible. CD, en este proyecto, automatiza el despliegue de una versión validada a Azure. El workflow diferencia Pull Request y push a main.

CI/CD de Admitionum

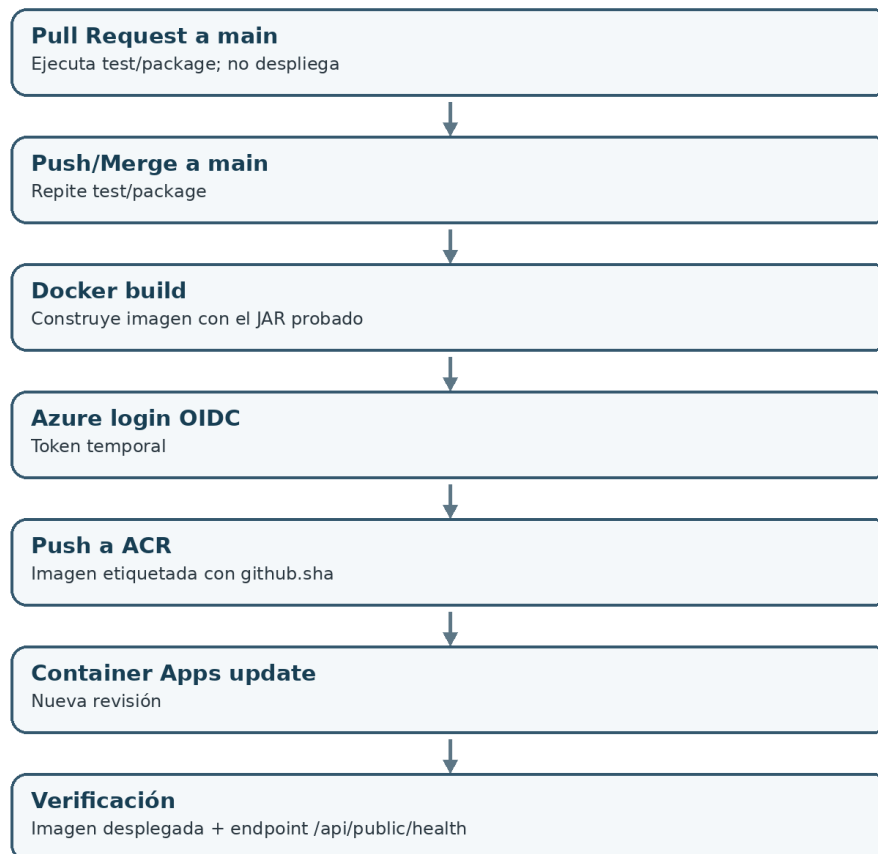


Figura 11. Pipeline de pruebas y despliegue.

Evento	Test/package	Build imagen	Push ACR	Deploy
pull_request -> main	Sí	No	No	No
push -> main	Sí	Sí	Sí	Sí
workflow_dispatch	Sí	Sí	Sí	Sí

El job `deploy` tiene `needs: test`; por tanto no se ejecuta si las pruebas fallan. Descarga el JAR probado como artifact, construye la imagen, autentica en Azure por OIDC, publica en ACR, actualiza Container Apps y verifica tanto la imagen como el endpoint de salud.

Error común

Pensar que un PR despliega producción. En el workflow actual el deploy está condicionado a `push` en main o `workflow\_dispatch`.

25. Azure OIDC

OIDC permite que GitHub Actions demuestre su identidad ante Azure y obtenga credenciales temporales. El workflow declara `id-token: write`, usa `azure/login` y proporciona identificadores mediante Secrets; no necesita almacenar un client secret permanente para el login federado.

OIDC entre GitHub y Azure

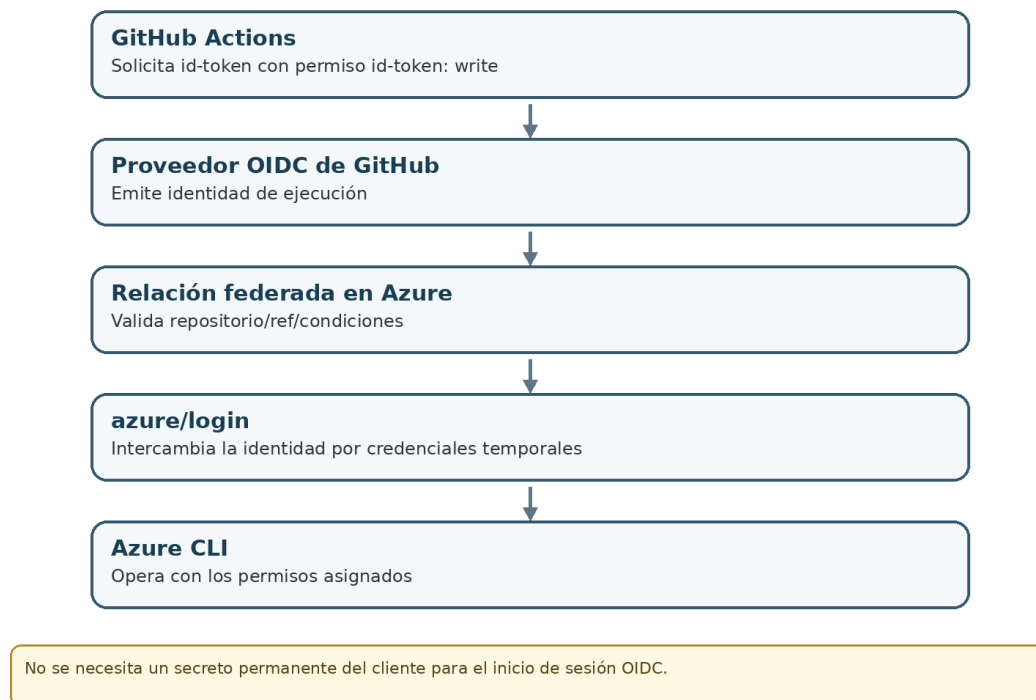


Figura 12. Intercambio de identidad OIDC.

Secret frente a Variable

Tipo	Ejemplos de uso	Visibilidad
GitHub Secret	Identificadores tratados como protegidos por política del proyecto; credenciales si existieran	Oculto en la interfaz/logs según mecanismos de GitHub.
GitHub Variable	Nombres de recursos, imagen, registry login server	Configuración no secreta.

`id-token: write` no concede permisos sobre Azure por sí mismo. La autorización efectiva depende de la identidad federada configurada y de los roles asignados en Azure. Esta separación es importante: autenticarse no equivale a tener permisos ilimitados.

26. Git y flujo de trabajo

Git proporciona historial, ramas y comparación de cambios. La trazabilidad muestra un trabajo por fases con ramas y Pull Requests. Para un principiante, la idea clave es separar “archivos en mi carpeta”, “cambios preparados”, “commit local”, “rama remota” y “main”.

Flujo Git y Pull Request



Figura 13. Flujo de rama a main.

Comprobaciones antes de commit

Comando	Qué responde
git status --short	Qué archivos están modificados, nuevos o sin seguimiento.
git diff	Cambios no preparados.
git diff --cached	Cambios preparados para commit.
git diff --check	Espacios problemáticos y errores de whitespace.
git log --oneline --decorate -n 10	Historial reciente y referencias.

La trazabilidad documenta un incidente pedagógicamente útil: apareció accidentalmente un archivo llamado `tatus --short`, probablemente por un comando mal escrito o una redirección inesperada. La lección es revisar `git status --short`, inspeccionar cualquier archivo extraño y eliminarlo antes de hacer commit si no pertenece al proyecto.

Cómo verificar

Antes de cada commit, el listado debe contener únicamente cambios intencionados. Un archivo con nombre inesperado es una señal para detener el commit y revisar.

27. Seguridad y secretos

La seguridad del proyecto combina controles de aplicación, manejo de secretos, separación de rutas y prácticas de despliegue. Ninguna medida aislada es suficiente: se trata de reducir exposición en varias capas.

Secretos

Credenciales de administrador y base de datos llegan por variables de entorno. GitHub no debe contener contraseñas, cadenas de conexión privadas ni tokens. `.dockerignore` excluye `.env`; del mismo modo, Git debe ignorar cualquier archivo local de secretos. Los ejemplos de esta guía son ficticios.

Limitación consciente del formulario anónimo

El endpoint general es público por diseño. Eso permite un QR común, pero también significa que no existe verificación de identidad del invitado, ni cuenta, ni email/SMS, ni deduplicación avanzada. El sistema crea un registro independiente por POST correcto. Es una decisión de alcance, no una propiedad que deba ocultarse.

AccessCode no es credencial

En el flujo actual general, AccessCode es un identificador interno generado para mantener compatibilidad con el modelo. No debe presentarse como contraseña del invitado.

28. Evolución histórica del proyecto

La trazabilidad v20 permite reconstruir el aprendizaje como una secuencia acumulativa. Cada fase incorporó una pieza y, cuando fue necesario, una fase “.1” corrigió un problema concreto sin rehacer el proyecto entero.

Bloques de aprendizaje

Fases	Bloque	Resultado
1-4	Fundamentos backend	Spring Boot, health, Invitation y consulta pública.
5-8	Persistencia y reglas	RsvpResponse, DTO, guardado/actualización y validación.
9-10.1	Frontend público	HTML/CSS, JavaScript y ampliación del límite a 20.
11-14	Administración	Security, API admin, panel y CSV.
15-17	Infraestructura	Azure SQL, Docker y Container Apps.
18	Automatización	GitHub Actions + OIDC + despliegue.
19	Portfolio	Documentación pública sin alterar función.
20	Evolución de requisitos	Registro general por enlace común y creación atómica de dos entidades.

La trazabilidad acumulativa conserva estados antiguos que ya no son la implementación vigente. Por eso una lectura histórica debe distinguir “en esa fase se decidió...” de “actualmente el código hace...”. El objetivo de esta guía es precisamente convertir esa secuencia en conocimiento transferible.

29. Fase 20 como caso práctico de evolución de requisitos

Fase 20 es un ejemplo de cómo un requisito aparentemente simple -usar un QR común- atraviesa varias capas. El cambio no se resuelve editando solo el HTML: modifica el contrato de API, la lógica de creación, la estrategia de pruebas, la documentación y la interpretación del AccessCode.

Antes y después

Aspecto	Antes	Actual
Entrada del invitado	URL/código individual	URL/QR común
Invitation	Preexistía antes de responder	Se crea durante el POST general
AccessCode	Visible en el viaje de invitado	Generado internamente; compatibilidad
Endpoint principal	GET/PUT por code	POST /api/public/registrations
RsvpResponse	Crear o actualizar sobre Invitation existente	Nueva respuesta asociada a nueva Invitation
Esquema SQL	Invitation 1 -> 0..1 RsvpResponse	Sin cambio

Por qué no cambia AdminService

El panel administrativo consume el mismo modelo persistente. Fase 20 no creó una tercera tabla ni un tipo alternativo de respuesta; creó los mismos objetos con otro punto de entrada. Esta reutilización es una señal de que la separación entre presentación, lógica y datos era suficientemente estable.

Pruebas que cerraron el riesgo

El ControllerTest comprueba 201 y validación; SecurityConfigTest garantiza acceso anónimo sin CSRF; FlowTest demuestra persistencia y visibilidad administrativa; TransactionTest prueba rollback. Juntas cubren el nuevo requisito desde HTTP hasta base de datos.

30. Cómo reproducir un proyecto semejante

Para reproducir un sistema parecido conviene avanzar de forma incremental y no empezar por Azure. Primero se consigue una aplicación local testeable; después se añade persistencia; luego API y frontend; a continuación seguridad y administración; finalmente contenedor, nube y automatización.

Ruta de aprendizaje recomendada

Etapas	Objetivo verificable
1	Crear Spring Boot y endpoint de salud.
2	Definir entidad y repositorio con prueba JPA.
3	Introducir DTO y servicio con reglas.
4	Exponer endpoint REST y probar con MockMvc.
5	Crear frontend mínimo y llamada fetch.
6	Añadir seguridad de sesión para administración.
7	Crear tests de integración y casos de error.
8	Separar perfiles local/test/prod.
9	Empaquetar JAR e imagen Docker.
10	Desplegar manualmente y después automatizar CI/CD.

Cada etapa debe terminar con una prueba reproducible. Esta disciplina evita que un error de infraestructura o de seguridad quede mezclado con un problema básico de modelo de datos.

31. Errores frecuentes y resolución de problemas

El diagnóstico mejora cuando se identifica primero la capa: herramienta, compilación, configuración, base de datos, HTTP, seguridad, contenedor, GitHub Actions o Azure. Saltar directamente a “cambiar código” suele empeorar un fallo de entorno.

Síntoma	Capa probable	Acción inicial
`java` no compatible	Entorno	Confirmar Java 25 y JAVA_HOME.
Maven no arranca	Wrapper/permisos	Usar mvnw/mvnw.cmd y revisar permiso.

Síntoma	Capa probable	Acción inicial
Puerto 8080 ocupado	Ejecución local	Identificar proceso o cambiar puerto temporalmente.
H2 conserva datos inesperados	Perfil local	Recordar que es file-based; revisar ./data.
Tests contaminan local	Perfil de test	Confirmar @ActiveProfiles("test") y configuración de test.
Validation 400	DTO/servicio	Leer `error.fields` o código de negocio.
Admin redirige a login	Security	Comprobar sesión y ROLE_ADMIN.
403 admin	Autorización/CSRF	Ver rol y token CSRF cuando corresponda.
Docker build no encuentra JAR	Build	Ejecutar Maven package/verify antes de docker build.
Container no conecta a SQL	Config/red	Variables, firewall, servidor y cifrado.
Actions falla antes de deploy	CI	Abrir job test; no investigar Azure todavía.
OIDC login falla	Federación	Revisar trust, subject/ref e IDs configurados.
Imagen desplegada no coincide	CD	Comparar Container App con IMAGE_REFERENCE.
Archivo raro en Git	Flujo Git	`git status --short`, inspeccionar y eliminar si accidental.

Método de diagnóstico

Reproduce el fallo en la capa más pequeña posible. Si `clean verify` falla, no tiene sentido depurar Container Apps todavía. Si tests pasan y solo falla prod, investiga configuración y entorno antes de reescribir lógica.

32. Glosario

Este glosario resume los términos más importantes usados a lo largo de la guía.

Término	Definición
API	Contrato por el que dos componentes se comunican.
HTTP	Protocolo de petición/respuesta usado por web y API.
Frontend	Código que se ejecuta en el navegador y presenta la interfaz.
Backend	Servidor que procesa solicitudes, reglas y datos.
Base de datos relacional	Almacén organizado en tablas relacionadas mediante claves.
Entidad JPA	Clase Java mapeada a una tabla o vista persistente.
Repository	Abstracción de acceso a datos de Spring Data.
DTO	Objeto de transporte que define datos de entrada o salida.

Término	Definición
Transacción	Conjunto de operaciones que se confirman o revierten como unidad.
Rollback	Reversión de una transacción fallida.
Spring Security	Framework de autenticación y autorización.
Autenticación	Comprobar la identidad de un usuario.
Autorización	Comprobar si ese usuario puede acceder a una acción.
CSRF	Ataque que intenta aprovechar una sesión autenticada para enviar una acción no deseada.
Dockerfile	Receta para construir una imagen Docker.
Imagen	Artefacto inmutable usado para crear contenedores.
Contenedor	Instancia en ejecución de una imagen.
CI	Integración continua: probar cambios automáticamente.
CD	Entrega/despliegue continuo: publicar versiones de forma automatizada.
OIDC	Protocolo de identidad usado aquí para federar GitHub Actions con Azure.
ACR	Azure Container Registry, registro de imágenes.
Azure Container Apps	Servicio gestionado que ejecuta contenedores.
AccessCode	Identificador de Invitation; en el flujo general actual se genera internamente.
UUID	Identificador aleatorio de gran espacio usado para generar AccessCode interno.
H2	Base de datos Java utilizada localmente y en pruebas.
Azure SQL	Servicio administrado de SQL Server utilizado en producción.

33. Conclusiones

Admissionum es especialmente útil como proyecto didáctico porque conecta conceptos que a menudo se aprenden por separado. Un mismo registro público permite estudiar HTTP, DTO, validación, transacciones, JPA, SQL, seguridad, pruebas, Docker, Azure y CI/CD.

La evolución hasta Fase 20 demuestra una idea importante: una arquitectura sencilla puede absorber cambios de requisitos si las responsabilidades están razonablemente separadas. El nuevo QR común necesitó un endpoint y lógica nueva, pero reutilizó entidades, repositorios, esquema y administración. Las pruebas de integración y rollback aportaron evidencia de que la modificación no introducía registros incompletos.

Idea final

El objetivo no es memorizar anotaciones, sino aprender a seguir el recorrido de los datos y verificar cada frontera: navegador -> HTTP -> controlador -> servicio -> transacción -> repositorio -> base de datos -> administración -> despliegue.

34. Apéndices

Los apéndices reúnen referencias rápidas, comandos, rutas, listas de verificación y fragmentos reales que resultan útiles durante el estudio o la reproducción del proyecto.

A. Matriz de archivos principales

Archivo	Qué estudiar
pom.xml	Versiones y dependencias.
.mvn/wrapper/maven-wrapper.properties	Maven 3.9.16 y wrapper 3.3.4.
InvitationService.java	Reglas, compatibilidad y transacción de Fase 20.
PublicRegistrationController.java	POST general y 201.
PublicInvitationController.java	GET/PUT por code de compatibilidad.
SaveRsvpRequest.java	Bean Validation.
Invitation.java / RsvpResponse.java	Mapeo JPA.
SecurityConfig.java	Rutas públicas/admin, sesión y CSRF.
database/schema.sql	PK, FK, UNIQUE y CHECK.
static/index.html + js/public-app.js	Formulario general.
static/admin/*	Panel administrativo.
application-*.properties	Perfiles.
Dockerfile	Runtime Java 25 y JAR.
.github/workflows/deploy.yml	Pipeline CI/CD y OIDC.

B. Referencia de comandos

Contexto	Comando	Resultado esperado
PowerShell	.\mvnw.cmd clean verify	BUILD SUCCESS
Git Bash	./mvnw clean verify	BUILD SUCCESS
Ejecutar local	.\mvnw.cmd spring-boot:run	Servidor 8080
Git	git status --short	Solo archivos intencionados
Git	git diff --check	Sin errores de whitespace
Docker	docker build -t admitionum .	Imagen creada
Docker	docker run --rm -p 8080:8080 admitionum	Contenedor escuchando en 8080

C. Variables de entorno

Variable	Entorno	Contenido
ADMIN_USERNAME	local/prod	Usuario administrativo; no hardcodear.

Variable	Entorno	Contenido
ADMIN_PASSWORD	local/prod	Contraseña administrativa; secreto.
SPRING_DATASOURCE_URL	prod	URL JDBC de Azure SQL.
SPRING_DATASOURCE_USERNAME	prod	Usuario SQL.
SPRING_DATASOURCE_PASSWORD	prod	Contraseña SQL; secreto.

D. Checklist de verificación local

- Java 25 activo.
- `clean verify` termina correctamente.
- Perfil local inicia con variables ADMIN_* ficticias.
- GET /api/public/health responde.
- Formulario `/` carga sin AccessCode.
- POST válido devuelve 201.
- Registro aparece en administración después de autenticarse.
- No hay secretos ni archivos extraños en `git status --short`.

E. Checklist de despliegue

- CI de PR verde antes del merge.
- Push a main ejecuta el job deploy.
- Login OIDC correcto.
- Imagen publicada en ACR con SHA.
- Container App usa exactamente esa imagen.
- Revisión saludable.
- `/api/public/health` devuelve status ok.
- Variables de producción disponibles sin exponer sus valores.

F. Fuentes del documento

Fuente primaria de implementación: repositorio público `Franer-Narf/Admissionum`, rama `main`, commit `5233f9d5b6c73c362af4cb2239852171b53fc796`. Se revisaron como mínimo `pom.xml`, Maven Wrapper, `src/main/java/`, `src/main/resources/`, pruebas de Fase 20, `database/schema.sql`, `Dockerfile`, `.dockerignore`, `.github/workflows/deploy.yml`, `README.md` y documentación de arquitectura, API, seguridad e instalación.

Fuente histórica: `traceability v20.txt`, archivo acumulativo aportado para este trabajo. Su naturaleza acumulativa permite reconstruir las fases 1-20 sin interpretar v20 como una única fase independiente. Cuando un bloque histórico describe un comportamiento sustituido, se presenta como evolución y no como estado actual.

G. Fragmento real de seguridad

```
.authorizeHttpRequests(authorize -> authorize
    .requestMatchers("/", "/index.html", "/css/**", "/js/**", "/api/public/**", "/error").permitAll()
    .requestMatchers("/admin/**", "/api/admin/**").hasRole("ADMIN")
    .anyRequest().denyAll()
)
.csrf(csrf -> csrf.ignoringRequestMatchers("/api/public/**"));
```

H. Fragmento real del flujo de Fase 20

Fragmento abreviado: se omiten argumentos únicamente para centrar la explicación en la transacción.

```
@Transactional
public SaveRsvpResponse registerPublicResponse(SaveRsvpRequest request) {
    // limpia y valida
    Invitation storedInvitation = invitationRepository.saveAndFlush(invitation);
    RsvpResponse response = new RsvpResponse(storedInvitation, ...);
    RsvpResponse storedResponse = rsvpResponseRepository.saveAndFlush(response);
    return new SaveRsvpResponse(true, "Tu respuesta se ha guardado correctamente.",
    storedResponse.getUpdatedAt());
}
```

I. Limitaciones conscientes

El proyecto no usa React, Angular, Vue, TypeScript, microservicios, JWT, Kubernetes, Azure Functions ni Microsoft Entra ID. El formulario general no verifica identidad individual ni evita por sí mismo múltiples registros de la misma persona. Estas ausencias no son errores ocultos: forman parte del alcance y de las decisiones del Documento Maestro.

APÉNDICE J. Cuaderno de estudio y verificación por capítulo

Este cuaderno amplía la guía principal con fichas de aprendizaje, conexiones entre componentes, laboratorios guiados y preguntas de comprobación. Su objetivo es que el lector pueda transformar una lectura pasiva en un proceso verificable. Las actividades no modifican el repositorio ni Azure: se centran en observar, razonar, ejecutar pruebas locales seguras y explicar el sistema con palabras propias.

Cada bloque parte del estado actual fijado en main y utiliza la trazabilidad v20 solo para contexto histórico. Cuando una actividad menciona credenciales, URLs o datos de invitado, los valores son ficticios. Antes de ejecutar cualquier comando, confirma que estás en una copia local del proyecto y que no estás operando contra producción.

J.1.1 - Ficha de aprendizaje: Portada

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Portada” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: RSVP, flujo público, datos estructurados, regla de negocio, QR común, compatibilidad.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: guía principal. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.1.2 - Conexiones y razonamiento: Portada

Primera conexión: RSVP se apoya en flujo público. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar portada, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
RSVP -> flujo público	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.1.3 - Laboratorio guiado: Portada

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre guía principal. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.2.1 - Ficha de aprendizaje: Cómo utilizar esta guía

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Cómo utilizar esta guía” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: RSVP, flujo público, datos estructurados, regla de negocio, QR común, compatibilidad.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: guía principal. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.2.2 - Conexiones y razonamiento: Cómo utilizar esta guía

Primera conexión: RSVP se apoya en flujo público. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar cómo utilizar esta guía, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
RSVP -> flujo público	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.2.3 - Laboratorio guiado: Cómo utilizar esta guía

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre guía principal. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.3.1 - Ficha de aprendizaje: El problema que resuelve Admitionum

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “El problema que resuelve Admitionum” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: RSVP, flujo público, datos estructurados, regla de negocio, QR común, compatibilidad.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: guía principal. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.3.2 - Conexiones y razonamiento: El problema que resuelve Admitionum

Primera conexión: RSVP se apoya en flujo público. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar el problema que resuelve admitionum, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
RSVP -> flujo público	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.3.3 - Laboratorio guiado: El problema que resuelve Admitionum

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre guía principal. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.3.4 - Preguntas de comprobación: El problema que resuelve Admitionum

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define RSVP con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre RSVP y flujo público?
- 3. ¿Qué error de principiante podría aparecer al trabajar con el problema que resuelve admitionum?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “El problema que resuelve Admitionum” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: RSVP, flujo público, datos estructurados, regla de negocio, QR común. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.4.1 - Ficha de aprendizaje: Visión general de la arquitectura

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Visión general de la arquitectura” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: monolito, frontend, backend, HTTP, API, capas, persistencia.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: README.md, docs/architecture.md. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.4.2 - Conexiones y razonamiento: Visión general de la arquitectura

Primera conexión: monolito se apoya en frontend. Ningún elemento importante de Admissionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar visión general de la arquitectura, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
monolito -> frontend	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.4.3 - Laboratorio guiado: Visión general de la arquitectura

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre README.md. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.4.4 - Preguntas de comprobación: Visión general de la arquitectura

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define monolito con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre monolito y frontend?
- 3. ¿Qué error de principiante podría aparecer al trabajar con visión general de la arquitectura?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Visión general de la arquitectura” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: monolito, frontend, backend, HTTP, API. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.5.1 - Ficha de aprendizaje: Preparación del entorno

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Preparación del entorno” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: Java 25, Maven Wrapper, Git, Docker, Azure CLI, entorno reproducible.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: pom.xml, .mvn/wrapper/maven-wrapper.properties. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.5.2 - Conexiones y razonamiento: Preparación del entorno

Primera conexión: Java 25 se apoya en Maven Wrapper. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar preparación del entorno, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
Java 25 -> Maven Wrapper	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.5.3 - Laboratorio guiado: Preparación del entorno

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre pom.xml. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo.

Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.5.4 - Preguntas de comprobación: Preparación del entorno

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define Java 25 con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre Java 25 y Maven Wrapper?
- 3. ¿Qué error de principiante podría aparecer al trabajar con preparación del entorno?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Preparación del entorno” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: Java 25, Maven Wrapper, Git, Docker, Azure CLI. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.6.1 - Ficha de aprendizaje: Estructura del proyecto

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Estructura del proyecto” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: controller, service, repository, model, dto, resources, tests.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: src/main/java/, src/main/resources/, src/test/java/. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.6.2 - Conexiones y razonamiento: Estructura del proyecto

Primera conexión: controller se apoya en service. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar estructura del proyecto, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
controller -> service	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.6.3 - Laboratorio guiado: Estructura del proyecto

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/java/`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.6.4 - Preguntas de comprobación: Estructura del proyecto

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define controller con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre controller y service?
- 3. ¿Qué error de principiante podría aparecer al trabajar con estructura del proyecto?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Estructura del proyecto” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: controller, service, repository, model, dto. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.7.1 - Ficha de aprendizaje: Modelo de datos

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Modelo de datos” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: Invitation, RsvpResponse, PK, FK, UNIQUE, CHECK, cardinalidad.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio:

src/main/java/nc/admitionum/model/Invitation.java,

src/main/java/nc/admitionum/model/RsvpResponse.java, database/schema.sql. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.7.2 - Conexiones y razonamiento: Modelo de datos

Primera conexión: Invitation se apoya en RsvpResponse. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar modelo de datos, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
Invitation -> RsvpResponse	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.7.3 - Laboratorio guiado: Modelo de datos

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/java/nc/admissionum/model/Invitation.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.7.4 - Preguntas de comprobación: Modelo de datos

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define Invitation con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre Invitation y RsvpResponse?
- 3. ¿Qué error de principiante podría aparecer al trabajar con modelo de datos?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Modelo de datos” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: Invitation, RsvpResponse, PK, FK, UNIQUE. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.8.1 - Ficha de aprendizaje: Spring Data JPA

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Spring Data JPA” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: JPA, Hibernate, entidad, repositorio, ORM, saveAndFlush.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `src/main/java/nc/admitionum/repository/`, `src/main/java/nc/admitionum/model/`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.8.2 - Conexiones y razonamiento: Spring Data JPA

Primera conexión: JPA se apoya en Hibernate. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar spring data jpa, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
JPA -> Hibernate	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.8.3 - Laboratorio guiado: Spring Data JPA

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/java/nc/admissionum/repository/`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.8.4 - Preguntas de comprobación: Spring Data JPA

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define JPA con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre JPA y Hibernate?
- 3. ¿Qué error de principiante podría aparecer al trabajar con spring data jpa?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Spring Data JPA” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: JPA, Hibernate, entidad, repositorio, ORM. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.9.1 - Ficha de aprendizaje: DTO y validación

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “DTO y validación” dentro de Admissionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: DTO, @Valid, Bean Validation, normalización, regla condicional.

En Admissionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio:

src/main/java/nc/admissionum/dto/publicapi/SaveRsvpRequest.java,

src/main/java/nc/admissionum/service/InvitationService.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admissionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.9.2 - Conexiones y razonamiento: DTO y validación

Primera conexión: DTO se apoya en @Valid. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar dto y validación, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
DTO -> @Valid	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.9.3 - Laboratorio guiado: DTO y validación

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/java/nc/admissionum/dto/publicapi/SaveRsvpRequest.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.9.4 - Preguntas de comprobación: DTO y validación

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define DTO con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre DTO y @Valid?
- 3. ¿Qué error de principiante podría aparecer al trabajar con dto y validación?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “DTO y validación” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: DTO, @Valid, Bean Validation, normalización, regla condicional. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.10.1 - Ficha de aprendizaje: API REST pública

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “API REST pública” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: REST, GET, POST, PUT, HTTP 201, JSON, contrato.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: docs/api.md, PublicRegistrationController.java, PublicInvitationController.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.10.2 - Conexiones y razonamiento: API REST pública

Primera conexión: REST se apoya en GET. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar api rest pública, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
REST -> GET	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.10.3 - Laboratorio guiado: API REST pública

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre docs/api.md. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.10.4 - Preguntas de comprobación: API REST pública

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define REST con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre REST y GET?
- 3. ¿Qué error de principiante podría aparecer al trabajar con api rest pública?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “API REST pública” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: REST, GET, POST, PUT, HTTP 201. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.11.1 - Ficha de aprendizaje: InvitationService y lógica de negocio

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “InvitationService y lógica de negocio” dentro de Admissionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: @Service, InvitationService, clean, UUID, GENERAL_MAX_GUESTS, compatibilidad.

En Admissionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio:

src/main/java/nc/admissionum/service/InvitationService.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admissionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.11.2 - Conexiones y razonamiento: InvitationService y lógica de negocio

Primera conexión: @Service se apoya en InvitationService. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar invitationservice y lógica de negocio, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
@Service -> InvitationService	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.11.3 - Laboratorio guiado: InvitationService y lógica de negocio

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/java/nc/admissionum/service/InvitationService.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.11.4 - Preguntas de comprobación: InvitationService y lógica de negocio

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define @Service con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre @Service y InvitationService?
- 3. ¿Qué error de principiante podría aparecer al trabajar con invitationservice y lógica de negocio?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “InvitationService y lógica de negocio” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: @Service, InvitationService, clean, UUID, GENERAL_MAX_GUESTS. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.12.1 - Ficha de aprendizaje: Transacciones y rollback

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Transacciones y rollback” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: @Transactional, atonicidad, commit, rollback, flush, consistencia.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: InvitationService.java, PublicRegistrationTransactionTest.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.12.2 - Conexiones y razonamiento: Transacciones y rollback

Primera conexión: @Transactional se apoya en atomicidad. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar transacciones y rollback, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
@Transactional -> atomicidad	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.12.3 - Laboratorio guiado: Transacciones y rollback

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `InvitationService.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.12.4 - Preguntas de comprobación: Transacciones y rollback

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define @Transactional con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre @Transactional y atomicidad?
- 3. ¿Qué error de principiante podría aparecer al trabajar con transacciones y rollback?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Transacciones y rollback” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: @Transactional, atomicidad, commit, rollback, flush. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.13.1 - Ficha de aprendizaje: Controladores Spring MVC

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Controladores Spring MVC” dentro de Admissionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: `@RestController`, `@RequestMapping`, `@RequestBody`, `ResponseEntity`, `MockMvc`.

En Admissionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `src/main/java/nc/admissionum/controller/`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admissionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.13.2 - Conexiones y razonamiento: Controladores Spring MVC

Primera conexión: `@RestController` se apoya en `@RequestMapping`. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar controladores spring mvc, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
<code>@RestController</code> -> <code>@RequestMapping</code>	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.13.3 - Laboratorio guiado: Controladores Spring MVC

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/java/nc/admissionum/controller/`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.13.4 - Preguntas de comprobación: Controladores Spring MVC

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define `@RestController` con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre `@RestController` y `@RequestMapping`?
- 3. ¿Qué error de principiante podría aparecer al trabajar con controladores spring mvc?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Controladores Spring MVC” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: `@RestController`, `@RequestMapping`, `@RequestBody`, `ResponseEntity`, `MockMvc`. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.14.1 - Ficha de aprendizaje: Frontend público

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Frontend público” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: HTML, CSS, JavaScript, DOM, evento, fetch, JSON.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `src/main/resources/static/index.html`, `src/main/resources/static/js/public-app.js`, `src/main/resources/static/css/styles.css`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.14.2 - Conexiones y razonamiento: Frontend público

Primera conexión: HTML se apoya en CSS. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar frontend público, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
HTML -> CSS	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.14.3 - Laboratorio guiado: Frontend público

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/resources/static/index.html`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.14.4 - Preguntas de comprobación: Frontend público

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define HTML con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre HTML y CSS?
- 3. ¿Qué error de principiante podría aparecer al trabajar con frontend público?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Frontend público” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: HTML, CSS, JavaScript, DOM, evento. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.15.1 - Ficha de aprendizaje: Panel administrativo

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Panel administrativo” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: dashboard, filtros, AdminService, CSV, estado administrativo.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: src/main/resources/static/admin/, AdminService.java, CsvExportService.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.15.2 - Conexiones y razonamiento: Panel administrativo

Primera conexión: dashboard se apoya en filtros. Ningún elemento importante de Admissionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar panel administrativo, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
dashboard -> filtros	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.15.3 - Laboratorio guiado: Panel administrativo

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/main/resources/static/admin/`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.15.4 - Preguntas de comprobación: Panel administrativo

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define dashboard con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre dashboard y filtros?
- 3. ¿Qué error de principiante podría aparecer al trabajar con panel administrativo?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Panel administrativo” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: dashboard, filtros, AdminService, CSV, estado administrativo. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.16.1 - Ficha de aprendizaje: Spring Security

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Spring Security” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: autenticación, autorización, ROLE_ADMIN, sesión, CSRF, denyAll.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: SecurityConfig.java, LocalSecurityConfig.java, SecurityConfigTest.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.16.2 - Conexiones y razonamiento: Spring Security

Primera conexión: autenticación se apoya en autorización. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar spring security, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
autenticación -> autorización	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.16.3 - Laboratorio guiado: Spring Security

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `SecurityConfig.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.16.4 - Preguntas de comprobación: Spring Security

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define autenticación con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre autenticación y autorización?
- 3. ¿Qué error de principiante podría aparecer al trabajar con spring security?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Spring Security” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: autenticación, autorización, ROLE_ADMIN, sesión, CSRF. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.17.1 - Ficha de aprendizaje: Gestión de errores

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Gestión de errores” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: `GlobalExceptionHandler`, 400, 404, 410, `VALIDATION_ERROR`, `INVALID_GUEST_COUNT`.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `GlobalExceptionHandler.java`, `src/main/java/nc/admitionum/exception/`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.17.2 - Conexiones y razonamiento: Gestión de errores

Primera conexión: GlobalExceptionHandler se apoya en 400. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar gestión de errores, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
GlobalExceptionHandler -> 400	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.17.3 - Laboratorio guiado: Gestión de errores

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `GlobalExceptionHandler.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.17.4 - Preguntas de comprobación: Gestión de errores

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define `GlobalExceptionHandler` con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre `GlobalExceptionHandler` y 400?
- 3. ¿Qué error de principiante podría aparecer al trabajar con gestión de errores?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Gestión de errores” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: `GlobalExceptionHandler`, 400, 404, 410, `VALIDATION_ERROR`. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.18.1 - Ficha de aprendizaje: Pruebas automatizadas

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Pruebas automatizadas” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: unitaria, MVC, JPA, seguridad, integración, regresión.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `src/test/java/nc/admitionum/`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.18.2 - Conexiones y razonamiento: Pruebas automatizadas

Primera conexión: unitaria se apoya en MVC. Ningún elemento importante de Admissionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar pruebas automatizadas, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
unitaria -> MVC	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.18.3 - Laboratorio guiado: Pruebas automatizadas

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `src/test/java/nc/admissionum/`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.18.4 - Preguntas de comprobación: Pruebas automatizadas

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define unitaria con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre unitaria y MVC?
- 3. ¿Qué error de principiante podría aparecer al trabajar con pruebas automatizadas?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Pruebas automatizadas” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: unitaria, MVC, JPA, seguridad, integración. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.19.1 - Ficha de aprendizaje: H2 y ejecución local

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “H2 y ejecución local” dentro de Admissionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: perfil local, H2 file, H2 memoria, ADMIN_USERNAME, puerto 8080.

En Admissionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: application.properties, application-local.properties. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admissionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.19.2 - Conexiones y razonamiento: H2 y ejecución local

Primera conexión: perfil local se apoya en H2 file. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar h2 y ejecución local, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
perfil local -> H2 file	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.19.3 - Laboratorio guiado: H2 y ejecución local

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `application.properties`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.19.4 - Preguntas de comprobación: H2 y ejecución local

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define perfil local con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre perfil local y H2 file?
- 3. ¿Qué error de principiante podría aparecer al trabajar con h2 y ejecución local?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “H2 y ejecución local” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: perfil local, H2 file, H2 memoria, ADMIN_USERNAME, puerto 8080. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.20.1 - Ficha de aprendizaje: Azure SQL Database

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Azure SQL Database” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: Azure SQL, JDBC, variables de entorno, ddl-auto=validate, firewall, cifrado.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: application-prod.properties, database/schema.sql. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.20.2 - Conexiones y razonamiento: Azure SQL Database

Primera conexión: Azure SQL se apoya en JDBC. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar azure sql database, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
Azure SQL -> JDBC	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.20.3 - Laboratorio guiado: Azure SQL Database

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `application-prod.properties`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.20.4 - Preguntas de comprobación: Azure SQL Database

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define Azure SQL con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre Azure SQL y JDBC?
- 3. ¿Qué error de principiante podría aparecer al trabajar con azure sql database?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Azure SQL Database” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: Azure SQL, JDBC, variables de entorno, ddl-auto=validate, firewall. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.21.1 - Ficha de aprendizaje: Docker

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Docker” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: Dockerfile, imagen, contenedor, JAR, eclipse-temurin, puerto 8080.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: Dockerfile, .dockerignore. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.21.2 - Conexiones y razonamiento: Docker

Primera conexión: Dockerfile se apoya en imagen. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar docker, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
Dockerfile -> imagen	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.21.3 - Laboratorio guiado: Docker

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre Dockerfile. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.21.4 - Preguntas de comprobación: Docker

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define Dockerfile con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre Dockerfile y imagen?
- 3. ¿Qué error de principiante podría aparecer al trabajar con docker?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Docker” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: Dockerfile, imagen, contenedor, JAR, eclipse-temurin. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.22.1 - Ficha de aprendizaje: Azure Container Registry

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Azure Container Registry” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: ACR, registry, tag, github.sha, push, imagen inmutable.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `.github/workflows/deploy.yml`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.22.2 - Conexiones y razonamiento: Azure Container Registry

Primera conexión: ACR se apoya en registry. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar azure container registry, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
ACR -> registry	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.22.3 - Laboratorio guiado: Azure Container Registry

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `.github/workflows/deploy.yml`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.22.4 - Preguntas de comprobación: Azure Container Registry

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define ACR con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre ACR y registry?
- 3. ¿Qué error de principiante podría aparecer al trabajar con azure container registry?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Azure Container Registry” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: ACR, registry, tag, github.sha, push. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.23.1 - Ficha de aprendizaje: Azure Container Apps

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Azure Container Apps” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: Container Apps, revisión, réplica, ingress, FQDN, health.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `.github/workflows/deploy.yml`, `docs/installation-guide.md`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.23.2 - Conexiones y razonamiento: Azure Container Apps

Primera conexión: Container Apps se apoya en revisión. Ningún elemento importante de Admissionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar azure container apps, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
Container Apps -> revisión	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.23.3 - Laboratorio guiado: Azure Container Apps

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `.github/workflows/deploy.yml`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.23.4 - Preguntas de comprobación: Azure Container Apps

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define Container Apps con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre Container Apps y revisión?
- 3. ¿Qué error de principiante podría aparecer al trabajar con azure container apps?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Azure Container Apps” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: Container Apps, revisión, réplica, ingress, FQDN. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.24.1 - Ficha de aprendizaje: GitHub Actions y CI/CD

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “GitHub Actions y CI/CD” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: GitHub Actions, CI, CD, artifact, needs, workflow_dispatch, health check.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `.github/workflows/deploy.yml`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.24.2 - Conexiones y razonamiento: GitHub Actions y CI/CD

Primera conexión: GitHub Actions se apoya en CI. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar github actions y ci/cd, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
GitHub Actions -> CI	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.24.3 - Laboratorio guiado: GitHub Actions y CI/CD

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `.github/workflows/deploy.yml`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.24.4 - Preguntas de comprobación: GitHub Actions y CI/CD

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define GitHub Actions con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre GitHub Actions y CI?
- 3. ¿Qué error de principiante podría aparecer al trabajar con github actions y ci/cd?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “GitHub Actions y CI/CD” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: GitHub Actions, CI, CD, artifact, needs. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.25.1 - Ficha de aprendizaje: Azure OIDC

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Azure OIDC” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: OIDC, id-token: write, federación, token temporal, azure/login, RBAC.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: `.github/workflows/deploy.yml`, `docs/security.md`. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.25.2 - Conexiones y razonamiento: Azure OIDC

Primera conexión: OIDC se apoya en id-token: write. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar azure oidc, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
OIDC -> id-token: write	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.25.3 - Laboratorio guiado: Azure OIDC

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `.github/workflows/deploy.yml`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.25.4 - Preguntas de comprobación: Azure OIDC

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define OIDC con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre OIDC y id-token: write?
- 3. ¿Qué error de principiante podría aparecer al trabajar con azure oidc?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Azure OIDC” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: OIDC, id-token: write, federación, token temporal, azure/login. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.26.1 - Ficha de aprendizaje: Git y flujo de trabajo

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Git y flujo de trabajo” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: branch, commit, push, pull request, merge, status, diff --check.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: .gitignore, trazabilidad v20. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.26.2 - Conexiones y razonamiento: Git y flujo de trabajo

Primera conexión: branch se apoya en commit. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar git y flujo de trabajo, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
branch -> commit	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.26.3 - Laboratorio guiado: Git y flujo de trabajo

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `.gitignore`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.26.4 - Preguntas de comprobación: Git y flujo de trabajo

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define branch con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre branch y commit?
- 3. ¿Qué error de principiante podría aparecer al trabajar con git y flujo de trabajo?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Git y flujo de trabajo” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: branch, commit, push, pull request, merge. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.27.1 - Ficha de aprendizaje: Seguridad y secretos

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Seguridad y secretos” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: secreto, variable de entorno, mínimo privilegio, frontera pública, CSV injection.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: SecurityConfig.java, .dockerignore, docs/security.md. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.27.2 - Conexiones y razonamiento: Seguridad y secretos

Primera conexión: secreto se apoya en variable de entorno. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar seguridad y secretos, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
secreto -> variable de entorno	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.27.3 - Laboratorio guiado: Seguridad y secretos

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `SecurityConfig.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.27.4 - Preguntas de comprobación: Seguridad y secretos

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define secreto con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre secreto y variable de entorno?
- 3. ¿Qué error de principiante podría aparecer al trabajar con seguridad y secretos?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Seguridad y secretos” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: secreto, variable de entorno, mínimo privilegio, frontera pública, CSV injection. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.28.1 - Ficha de aprendizaje: Evolución histórica del proyecto

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Evolución histórica del proyecto” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: fases, trazabilidad acumulativa, correcciones .1, aprendizaje incremental.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: traceability v20.txt. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.28.2 - Conexiones y razonamiento: Evolución histórica del proyecto

Primera conexión: fases se apoya en trazabilidad acumulativa. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar evolución histórica del proyecto, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
fases -> trazabilidad acumulativa	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.28.3 - Laboratorio guiado: Evolución histórica del proyecto

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `traceability v20.txt`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.28.4 - Preguntas de comprobación: Evolución histórica del proyecto

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define fases con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre fases y trazabilidad acumulativa?
- 3. ¿Qué error de principiante podría aparecer al trabajar con evolución histórica del proyecto?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Evolución histórica del proyecto” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: fases, trazabilidad acumulativa, correcciones .1, aprendizaje incremental. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.29.1 - Ficha de aprendizaje: Fase 20 como caso práctico de evolución de requisitos

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Fase 20 como caso práctico de evolución de requisitos” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: Fase 20, cambio de requisito, QR común, creación automática, reutilización, rollback.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: InvitationService.java, PublicRegistrationController.java, PublicRegistrationFlowTest.java, PublicRegistrationTransactionTest.java. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.29.2 - Conexiones y razonamiento: Fase 20 como caso práctico de evolución de requisitos

Primera conexión: Fase 20 se apoya en cambio de requisito. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar fase 20 como caso práctico de evolución de requisitos, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
Fase 20 -> cambio de requisito	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.29.3 - Laboratorio guiado: Fase 20 como caso práctico de evolución de requisitos

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre `InvitationService.java`. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.29.4 - Preguntas de comprobación: Fase 20 como caso práctico de evolución de requisitos

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define Fase 20 con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre Fase 20 y cambio de requisito?
- 3. ¿Qué error de principiante podría aparecer al trabajar con fase 20 como caso práctico de evolución de requisitos?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Fase 20 como caso práctico de evolución de requisitos” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: Fase 20, cambio de requisito, QR común, creación automática, reutilización. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.30.1 - Ficha de aprendizaje: Cómo reproducir un proyecto semejante

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Cómo reproducir un proyecto semejante” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: incremental, prueba verificable, separación de capas, local primero, nube después.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: todo el repositorio por capas. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.30.2 - Conexiones y razonamiento: Cómo reproducir un proyecto semejante

Primera conexión: incremental se apoya en prueba verificable. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar cómo reproducir un proyecto semejante, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
incremental -> prueba verificable	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.30.3 - Laboratorio guiado: Cómo reproducir un proyecto semejante

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre todo el repositorio por capas. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.30.4 - Preguntas de comprobación: Cómo reproducir un proyecto semejante

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define incremental con tus palabras y explica dónde aparece en Admissionum.
- 2. ¿Qué relación existe entre incremental y prueba verificable?
- 3. ¿Qué error de principiante podría aparecer al trabajar con cómo reproducir un proyecto semejante?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Cómo reproducir un proyecto semejante” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: incremental, prueba verificable, separación de capas, local primero, nube después. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.31.1 - Ficha de aprendizaje: Errores frecuentes y resolución de problemas

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Errores frecuentes y resolución de problemas” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: diagnóstico, capa, reproducción mínima, logs, configuración, entorno.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: tests, logs, application-*.properties, workflow. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.31.2 - Conexiones y razonamiento: Errores frecuentes y resolución de problemas

Primera conexión: diagnóstico se apoya en capa. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar errores frecuentes y resolución de problemas, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
diagnóstico -> capa	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.31.3 - Laboratorio guiado: Errores frecuentes y resolución de problemas

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre tests. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo.

Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.31.4 - Preguntas de comprobación: Errores frecuentes y resolución de problemas

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define diagnóstico con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre diagnóstico y capa?
- 3. ¿Qué error de principiante podría aparecer al trabajar con errores frecuentes y resolución de problemas?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Errores frecuentes y resolución de problemas” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: diagnóstico, capa, reproducción mínima, logs, configuración. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.32.1 - Ficha de aprendizaje: Glosario

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Glosario” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: API, HTTP, JPA, DTO, transacción, Docker, OIDC.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: todo el documento. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.32.2 - Conexiones y razonamiento: Glosario

Primera conexión: API se apoya en HTTP. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar glosario, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
API -> HTTP	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.32.3 - Laboratorio guiado: Glosario

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre todo el documento. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.32.4 - Preguntas de comprobación: Glosario

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define API con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre API y HTTP?
- 3. ¿Qué error de principiante podría aparecer al trabajar con glosario?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Glosario” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: API, HTTP, JPA, DTO, transacción. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.33.1 - Ficha de aprendizaje: Conclusiones

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Conclusiones” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: transferencia de aprendizaje, recorrido de datos, verificación, evolución.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: capítulos 3-31. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.33.2 - Conexiones y razonamiento: Conclusiones

Primera conexión: transferencia de aprendizaje se apoya en recorrido de datos. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar conclusiones, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
transferencia de aprendizaje -> recorrido de datos	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.33.3 - Laboratorio guiado: Conclusiones

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre capítulos 3-31. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.33.4 - Preguntas de comprobación: Conclusiones

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define transferencia de aprendizaje con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre transferencia de aprendizaje y recorrido de datos?
- 3. ¿Qué error de principiante podría aparecer al trabajar con conclusiones?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Conclusiones” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: transferencia de aprendizaje, recorrido de datos, verificación, evolución. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.34.1 - Ficha de aprendizaje: Apéndices

Objetivo de esta ficha. Al terminar este bloque deberías poder explicar qué papel desempeña “Apéndices” dentro de Admitionum sin limitarte a repetir nombres de clases o herramientas. La meta es conectar intención, implementación y evidencia. Para ello, utiliza como vocabulario de referencia: referencia, checklist, rutas, comandos, fuentes.

En Admitionum, este tema no aparece aislado. Se relaciona con el recorrido completo que comienza en una necesidad del usuario y termina en un estado verificable del sistema. Pregúntate siempre tres cosas: qué dato entra, qué decisión toma esta capa y qué dato sale. Ese enfoque evita memorizar anotaciones sin comprender por qué existen.

Archivos o zonas que conviene abrir durante el estudio: apéndices y fuentes. No es necesario entender cada línea en la primera lectura. Localiza primero los nombres que ya conoces por la guía, después identifica dependencias y, por último, sigue una llamada real desde su origen hasta su consecuencia.

Pregunta	Respuesta esperada al finalizar
¿Qué problema resuelve este componente?	Una frase funcional, no solo tecnológica.
¿Dónde está implementado?	Ruta o grupo de archivos concreto.
¿Qué recibe?	Petición, DTO, entidad, configuración o artefacto.
¿Qué produce?	Respuesta, persistencia, imagen, despliegue o evidencia.
¿Cómo se verifica?	Prueba, comando o estado observable.

Criterio de dominio

Si puedes explicar el tema usando un ejemplo de Admitionum y también trasladarlo a otra aplicación pequeña, has entendido el concepto más allá de este repositorio.

J.34.2 - Conexiones y razonamiento: Apéndices

Primera conexión: referencia se apoya en checklist. Ningún elemento importante de Admitionum funciona de forma autónoma. Incluso una clase pequeña suele existir porque otra capa necesita un contrato estable. Dibuja mentalmente una flecha de entrada y otra de salida para cada término.

Segunda conexión: el proyecto diferencia configuración, lógica y datos. Al estudiar apéndices, marca qué parte pertenece a código Java, qué parte es configuración, qué parte es persistencia y qué parte solo existe en tiempo de despliegue. Esta clasificación es especialmente útil para no intentar resolver con código un problema de entorno.

Tercera conexión: las pruebas son documentación ejecutable. Busca una prueba relacionada y formula antes de leerla qué comportamiento crees que debería proteger. Después compara tu predicción con las aserciones. Si no hay una prueba directa, identifica qué prueba de integración cubre indirectamente el componente.

Cuarta conexión: el comportamiento histórico puede ser distinto del actual. Si la trazabilidad menciona un flujo anterior, etiqueta mentalmente ese texto como “historia”. La decisión de precedencia de esta guía es clara: código actual de main, documentación actual, trazabilidad más reciente y, por último, bloques históricos más antiguos.

Relación	Pregunta para razonar
referencia -> checklist	¿Qué pasaría si el primero entregara un dato inválido?
Código -> prueba	¿Qué regresión detectaría la prueba?
Local -> producción	¿Qué cambia por perfil y qué debería permanecer igual?
Historia -> estado actual	¿Es una evolución o una contradicción real?

J.34.3 - Laboratorio guiado: Apéndices

Laboratorio seguro. Trabaja sobre una copia local y no hagas commits mientras exploras. El objetivo no es modificar el proyecto, sino comprobar que puedes localizar evidencias y seguir el flujo. Empieza con ``git status --short``; si ya existen cambios que no reconoces, no continúes hasta entenderlos.

Paso 1. Abre apéndices y fuentes. Identifica al menos tres símbolos que correspondan a los conceptos del capítulo. Paso 2. Si hay una ruta HTTP, localiza quién la declara y qué servicio invoca. Si hay una entidad o tabla, localiza la clave y las restricciones. Si es infraestructura, localiza el punto exacto del workflow o configuración donde se referencia.

Paso 3. Busca una prueba relacionada y lee el nombre del método antes del cuerpo. Explica en lenguaje natural qué riesgo intenta cubrir. Paso 4. Ejecuta ``clean verify`` solo si tu entorno está preparado; una ejecución correcta proporciona una señal amplia de regresión, aunque no sustituye la inspección del comportamiento concreto.

Paso 5. Escribe una nota con dos columnas: “hecho observable” e “interpretación”. Ejemplo: hecho observable: ``POST /api/public/registrations`` devuelve 201 en la prueba. Interpretación: el controlador representa una creación exitosa. Separar evidencia de interpretación mejora la calidad del análisis técnico.

Comprobación	Resultado que debes poder describir
Ubicación	Sé en qué archivo/capa vive el comportamiento.
Dependencia	Sé qué componente llama a cuál.
Persistencia o salida	Sé qué estado cambia o qué respuesta se produce.
Prueba	Sé qué test proporciona evidencia.
Límite	Sé qué NO demuestra esa prueba.

No modifiques producción

Los laboratorios de este cuaderno no requieren ``az containerapp update``, pushes, merges ni cambios de recursos Azure. La observación y las pruebas locales son suficientes.

J.34.4 - Preguntas de comprobación: Apéndices

Responde sin mirar la guía y después contrasta. No busques frases exactas; busca relaciones causales. Una respuesta buena incluye el componente, su responsabilidad, la evidencia y al menos una limitación.

- 1. Define referencia con tus palabras y explica dónde aparece en Admitionum.
- 2. ¿Qué relación existe entre referencia y checklist?
- 3. ¿Qué error de principiante podría aparecer al trabajar con apéndices?
- 4. ¿Qué prueba o comando usarías para obtener evidencia sin modificar producción?
- 5. ¿Qué dato o detalle no debería exponerse públicamente en este contexto?
- 6. ¿Qué parte del comportamiento podría cambiar por perfil local/test/prod?
- 7. ¿Qué afirmación histórica de la trazabilidad podría confundirse con el estado actual?
- 8. Si este componente desapareciera, ¿qué fallo funcional u operativo observarías primero?

Actividad de síntesis. Explica “Apéndices” a una persona que conoce Java de forma básica, usando como máximo cinco conceptos: referencia, checklist, rutas, comandos, fuentes. Después vuelve a explicarlo a una persona no técnica centrándote exclusivamente en el problema que resuelve. Si ambas explicaciones son coherentes, has separado correctamente mecanismo y finalidad.

Autocomprobación

Puedes pasar al capítulo siguiente cuando puedas justificar tus respuestas con una ruta de archivo, una prueba o una observación directa del sistema, no solo con memoria del texto.

J.35 - Checklist global de dominio

- Puedo dibujar el flujo QR/enlace común -> formulario -> POST -> servicio -> transacción -> dos entidades -> administración.
- Puedo explicar por qué AccessCode sigue existiendo sin ser el flujo público principal.
- Puedo diferenciar DTO, entidad y tabla SQL con ejemplos del proyecto.
- Puedo explicar por qué la validación del frontend no es suficiente.
- Puedo describir el rollback y citar la prueba que lo demuestra.
- Puedo distinguir autenticación, autorización y CSRF.
- Puedo distinguir Dockerfile, imagen y contenedor.
- Puedo explicar PR CI frente a deploy por push a main.
- Puedo explicar OIDC sin confundir autenticación con permisos Azure.
- Puedo usar la trazabilidad como historia sin dejar que sustituya al código actual.

Cuando todos los puntos anteriores puedan explicarse con evidencia concreta, el proyecto deja de ser una secuencia de instrucciones y se convierte en un modelo mental reutilizable para otras aplicaciones.